Исследование: перенос gMART на Google ADK

Oct 26, 2026 · @Leon

Резюме

Перенос на Google ADK 2.13 оправдан и выполним поэтапно без изменения контрактов фронта и A2A. Пилот на restriction-агенте в 9 сценариях даёт тот же поток событий, те же запросы к LLM и те же A2A-кадры, что и текущий сервис. Полная миграция — примерно 16–20 недель одного разработчика.

Цели и ответ на каждую:

- **Управляемый workflow.** Да. Графовый `Workflow` из ADK 2.x ложится на пайплайны gMART естественнее классического агента с инструментами. Ветвления, петли repair и retry становятся объявлениями графа, который проверяется при создании.
- **Управляемый harness.** Да. Callbacks и plugins дают одно место для параметров запроса, контекста, аудита и учёта. В пилоте `AuditPlugin` видит каждый вызов модели и каждый узел.
- **Локальные модели через OpenAI-совместимые эндпоинты.** Да, но через свой `GmartLlm` поверх текущих адаптеров. Встроенный `OpenAILlm` принудительно включает strict-схему, не умеет `extra_body` и конфликтует по версии `openai`.
- **Контракты фронта и A2A.** Сохраняются полностью. Поток событий остаётся прежним, `python-a2a` и карточки Synapse не меняются.
- **Отказ от dev UI.** Да, он не обязателен: `Runner` встраивается в текущий FastAPI.
- **Рост точности.** Сам по себе переход её не даст: при тех же промптах и параметрах ответы модели те же. Выигрыш в том, что узлы проверки, петли самопроверки и оценку на уровне узла станет дешевле добавлять и измерять.

Рекомендация. Начать с этапов 0–1: общий слой и restriction в проде за флагом, примерно 4 недели. Решение о полной миграции принимать после прогона `evaluation/orchestrator` на реальных моделях и замера задержки с Redis-сессиями.

Текущая архитектура gMART и её соответствие ADK

gMART — это детерминированные пайплайны, в которых LLM играет роль планировщика, а не автономного агента. Модель возвращает JSON по Pydantic-схеме, код его проверяет и сам вызывает MCP-инструменты. Нативного function calling нет нигде. Такая модель управления близка к графовому `Workflow` из ADK 2.x и далека от классического `LlmAgent` с инструментами.

Объём: около 65 тыс. строк в `src/`, из них около 43 тыс. в `src/agents/services`, плюс около 37 тыс. строк тестов в 165 файлах. Вызовов LLM — 26, в 19 файлах, прямых вызовов MCP — 39.

Слой gMART	Как устроен сейчас	Аналог в ADK 2.13
Модели (`model_clients/`)	Свой `BaseLlmAdapter` (Ollama и OpenAI-совместимые серверы), ответы в формате Ollama, `llm_pace`, `context_budget`, `llm_usage`	`OpenAILlm(base_url=…)`, `FallbackModel`, before/after model callbacks
Пайплайны (`services/*`)	Async-генераторы, которые отдают события-словари (`status`, `chunk`, `feature_collection`, `tool_call`, `clarification`)	`Workflow`: function-узлы, `LlmAgent`-узлы, маршруты, циклы
Самопроверка плана	Цикл в коде: запрос → валидация → канонизация → семантическая проверка → repair → при неудаче уточнение	Условные рёбра (петля repair) и `RetryConfig` на узле
Чекпоинты и возобновление	`PipelineStateStore` в Redis, ключи `PipelineStep.*`, повтор буфера событий при переподключении	Восстановление `Workflow` из событий сессии, `BaseSessionService`
MCP	`fastmcp.Client`, обёртки `*McpClient`, вызовы инструментов из кода	`McpToolset` (для LLM) или прямой вызов из function-узла
A2A	`python-a2a` (`A2AServer`), свой `A2ATaskStore`, карточки агентов доработаны под Synapse	`A2aAgentExecutor` и `to_a2a` на `a2a-sdk` 1.x

Слой gMART	Как устроен сейчас	Аналог в ADK 2.13
Фронтенд	REST и SSE поверх того же потока событий	Не входит в ADK, нужен адаптер

Главное наблюдение для сохранения контрактов: REST/SSE и A2A-исполнитель (`a2a/executor.py`, `_pipeline_item_to_event`) читают один и тот же внутренний поток событий. Значит, если новый движок будет выдавать тот же поток, внешние контракты останутся прежними.

Возможности ADK 2.13, важные для целей

В ADK 2.x появился графовый `Workflow`, именно он даёт управляемый workflow. В 1.x были только `SequentialAgent`, `ParallelAgent` и `LoopAgent`. Исследована версия `google-adk==2.13.0`: исходники, гайды из `docs/guides` и примеры из `contributing/samples` на теге `v2.13.0`.

Цель	Механизм ADK	Что даёт gMART	Ограничения
Управляемый workflow	`Workflow(edges=[…])`: цепочки, fan-out, маршруты `{route: node}`, `DEFAULT_ROUTE`, условные циклы, `JoinNode`, вложенные workflow, `ctx.run_node()`	Пайплайн описан декларативно, граф проверяется при создании. Ветвления вместо `if … return`	Безусловные циклы запрещены. Два терминальных узла с выходом — ошибка
Устойчивость шагов	`RetryConfig` и `timeout` на любом узле, `NodeTimeoutError`	Замена ручных `_retries` и таймаутов MCP	—
Возобновление	Уже выполненные узлы восстанавливаются из событий сессии и не запускаются повторно. Есть `RedisSessionService` (extra `redis`)	Может заменить чекпоинты `PipelineStateStore`	Восстановление идёт по сессии, а не по `request_id`. Ключи придётся сопоставить
Уточняющие вопросы (HITL)	`RequestInput`, `require_confirmation` на узле или инструменте	`NEEDS_CLARIFICATION` становится паузой с возобновлением, а не концом запроса	Меняет семантику диалога, фронту нужно то же событие `clarification`

Цель	Механизм ADK	Что даёт gMART	Ограничения
Управляемые параметры LLM	`LlmAgent`: `instruction` (в том числе функция), `output_schema`, `output_key`, `include_contents`, `generate_content_config` (temperature, max tokens), `mode`	Один декларативный объект на каждый LLM-шаг вместо разбросанных `options={…}`	Точность сама по себе не растёт: параметры те же, меняется только удобство их настройки и аудита
Harness	Plugins (`BasePlugin`): хуки до и после run, agent, model, tool, `on_event`, `on_*_error`. Готовые `ReflectAndRetryToolPlugin`, `LoggingPlugin`, `ContextFilterPlugin`	Одно место для редакции чувствительных данных, учёта токенов, бюджета контекста, лимита частоты и аудита вызовов MCP	Текущие `llm_pace` и `context_budget` придётся перенести в плагины или в модель
Локальные модели	`OpenAILlm(model, base_url, api_key)` без LiteLLM: стриминг, tools, `response_format=json_schema`. Есть `LiteLlm` и `FallbackModel`	vLLM и любые OpenAI-совместимые провайдеры	Всегда `strict: true`: все поля становятся обязательными. Нет `extra_body`, то есть нет `chat_template_kwargs` (enable_thinking) — а это текущий механизм `think=False`. Нужен свой `BaseLlm`
MCP	`McpToolset` и `McpTool`, инструмент как узел графа, `ReflectAndRetryToolPlugin`	Стандартное подключение к IDU MCP и фильтр по тегам	В restriction инструменты вызывает код. Для него текущий `IduMcpClient` внутри function-узла проще и надёжнее
A2A	`A2aAgentExecutor`, `to_a2a`, `AgentCardBuilder`, `RemoteA2aAgent`	Стандартный сервер при полном переходе	Идёт с `a2a-sdk` 1.2.3 (протокол 1.0, режим совместимости с 0.3). gMART сейчас отдаёт 0.3.0 с доработками под Synapse
Оценка качества	`AgentEvaluator` (pytest), `LocalEvalService`, метрики по траектории и расходу	Можно написать свои `Evaluator` поверх `evaluation/`	Правила проверки для геометрии придётся писать самим

Цель	Механизм ADK	Что даёт gMART	Ограничения
Без dev UI	`Runner` и `App` встраиваются в свой FastAPI. `adk web` и `get_fast_api_app` не обязательны	Текущие контроллеры остаются	—

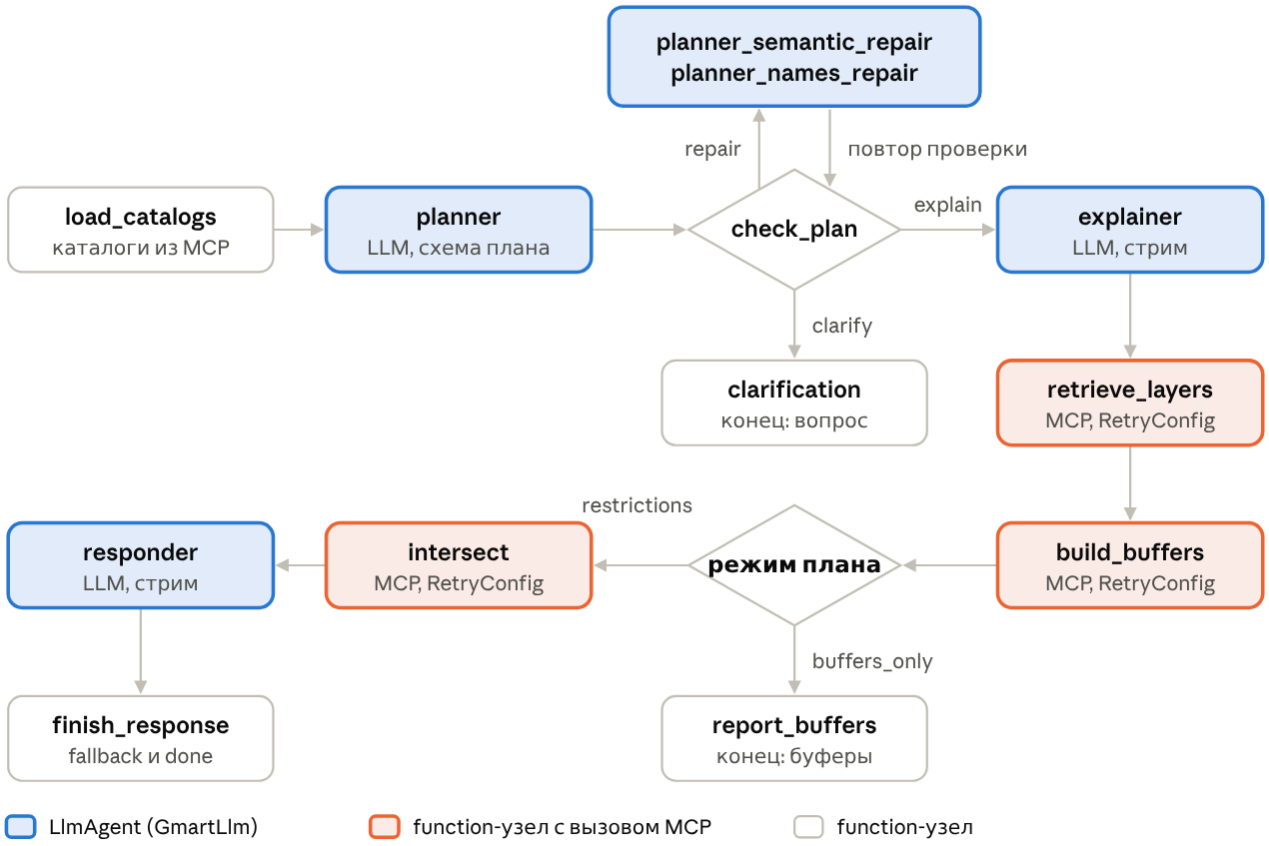
Вывод по точности. ADK не делает модель точнее сам по себе. Он даёт структуру, в которой проще улучшать точность: отдельные узлы проверки, явные петли repair, трассировку и оценку на уровне узла. В restriction логика самопроверки уже есть, так что выигрыш в основном в прозрачности и управляемости, а не в метриках. Это нужно измерить на `evaluation/`.

Пилот: restriction-агент на ADK

Прототип воспроизводит основной путь restriction на `Workflow` из ADK 2.13 и на моках ведёт себя байт в байт как текущий `RestrictionParserService`. Совпадают события для фронта, запросы к LLM и A2A-кадры. Код лежит в ветке `research/adk-restriction-pilot`, папка `research/adk_pilot/`, `src/` не тронут. Объём: около 1000 строк кода и около 720 строк тестов.

В пилот не вошли: ветка compliance, поиск в NormGraph, создание и запись чата, повтор событий при переподключении из Redis.

План проходит проверку и repair в графе, затем ветвится по режиму



restriction_workflow · 17 узлов, четыре служебных announce-узла не показаны

Все ветвления, которые сейчас записаны в коде `if … return`, стали маршрутами графа. Петли repair ограничены состоянием `plan_phase` и выполняются в том же порядке, что и в `RestrictionPlanBuilder.build_plan`.

Результаты проверок (все на моках)

Проверка	Результат
Эквивалентность, 9 сценариев: пересечения, только буферы, уточнение, семантический repair (успешный и нет), repair имён, ремонт JSON, история чата, fallback пустого ответа	Поток событий и все запросы к LLM (`messages`, `options`, `format`, `think`, `stream`) совпадают с текущим сервисом
A2A: неизменённый `RestrictionAgentExecutor` поверх ADK-сервиса	Те же кадры и то же состояние задачи (без учёта `messageId` и `timestamp`)

Проверка	Результат
Retry MCP-шага: первый вызов падает, `RetryConfig(max_attempts=2)`	Шаг повторяется, события не дублируются, поток равен потоку без сбоя
Harness: `AuditPlugin`	Видит все 4 вызова модели и порядок узлов, включая петлю repair
Два параллельных запроса на одном workflow	Каждый видит свой MCP-клиент (`ContextVar`)
Существующие тесты с ADK в окружении	2020 unit-тестов проходят, вместе с пилотом 2042
Накладные расходы движка (LLM и MCP мгновенные, 50 прогонов)	Медиана 50,7 мс против 5,5 мс у текущего движка, то есть примерно +45 мс на запрос. Сборка сервиса и графа — 20 мс, один раз

Точность на реальных моделях не замерялась: нет доступа к эндпоинту. Раз запросы к LLM совпадают дословно, точность пилота должна совпасть с текущей. Выигрыш появится только от изменений, которые граф упрощает.

Решения дизайна

1. **Своя модель `GmartLlm(BaseLlm)` поверх текущего `BaseLlmAdapter`** вместо `OpenAILlm` и `LiteLlm`. Сохраняются `think`, схема без strict-режима, Ollama и vLLM, а заодно уходит конфликт версий `openai`.
2. **Параметры каждого LLM-шага описаны декларативно** в `GmartGenerateContentConfig`: temperature, max tokens, `think`, `num_ctx`, число попыток ремонта JSON. Запрошенные модель и температура подставляются callback'ом `request_params`.
3. **Доменная логика переиспользована, в граф перенесено только управление.** Узел `check_plan` повторяет порядок `build_plan`: семантический repair, затем repair имён, затем уточнение. Repair-вызовы оформлены отдельными `LlmAgent` и связаны условными рёбрами.
4. **Контракт событий.** Function-узлы кладут прежние события в `Event.custom_metadata`, текст `explainer` и `responder` стримится как `chunk`. `LegacyEventAdapter` собирает из этого исходный поток.
5. **Один статичный workflow на сервис.** Зависимости запроса, например MCP-клиент, передаются через `ContextVar`, данные — через состояние сессии.

6. **MCP вызывается из function-узлов через текущий `RestrictionToolExecutor`, а не через `McpToolset`.** Модель не выбирает инструменты, и это намеренно: так пайплайн остаётся детерминированным. Статусы вынесены в отдельные `announce_*`-узлы, чтобы retry не дублировал события.

Сохранение контрактов фронта и A2A

Оба контракта можно сохранить без изменений, и пилот это показывает. Правило такое: ADK меняет движок внутри, наружу по-прежнему выдаётся тот же поток событий. Адаптеры вокруг этого потока остаются на месте.

- **Фронтенд (REST/SSE).** Контроллеры вызывают `run_restriction_execution_pipline`. `AdkRestrictionService` реализует тот же метод с теми же keyword-аргументами и отдаёт те же словари. Переключение сводится к замене одной зависимости в `dependencies.py` и может стоять за флагом.
- **A2A.** Текущий стек `python-a2a`: протокол 0.3.0, свой `A2ATaskStore`, карточки под Synapse. Он остаётся и просто получает ADK-сервис вместо текущего. Контрактный тест прогоняет настоящий `RestrictionAgentExecutor` на обоих движках и получает одинаковые кадры.
- **Встроенный A2A из ADK** (`A2aAgentExecutor`, `to_a2a`) применять пока не стоит. Он идёт с `a2a-sdk` 1.2.3 и протоколом 1.0, совместимость с 0.3 даётся режимом совместимости или закреплением `a2a-sdk` 0.3.x. Карточки под Synapse и формат артефактов пришлось бы воссоздавать. Это отдельный шаг после миграции движка, и делать его стоит только если Synapse перейдёт на A2A 1.0.
- **Находка в текущем коде.** `RestrictionAgentExecutor._pipeline_item_to_event` отбрасывает события `clarification`. В итоге A2A-клиент получает задачу в статусе `completed` без самого уточняющего вопроса. Пилот воспроизводит это поведение как часть контракта. Исправлять стоит отдельно, например через состояние `input-required`.
- **История и переподключение.** Сейчас события буферизуются в Redis по `request_id`, а клиент повторяет их при переподключении. В ADK ту же роль играет сессия: `session_id = request_id`, `RedisSessionService` или `DatabaseSessionService`. Повтор событий придётся переписать на чтение событий сессии через тот же `LegacyEventAdapter`. В пилоте это не сделано.

Риски и открытые вопросы

Главные риски — скорость, с которой меняется ADK 2.x, и размер сессий с GeoJSON. Оба решаемы.

Риск	Влияние	Что делать
ADK 2.x меняется быстро: с 2.0.0 до 2.13.0 вышло 20 релизов (14 минорных версий). В 2.0 были ломающие изменения API агентов, модели событий и схемы сессий	Обновления могут ломать поведение	Закрепить точную версию, держать тесты эквивалентности и A2A в CI, обновляться планово
ADK оптимизирован под Gemini. Для локальных моделей нужен свой `GmartLlm`	Свой код модели придётся поддерживать (около 200 строк)	Оставить тонкий мост над текущими адаптерами. Tools добавлять, только когда потребуется
Extra `google-adk[openai]` требует `openai<3`, а gMART — `openai>=3.3.1`	С extra зависимости не разрешаются	Ставить `google-adk[mcp]`, модель через `GmartLlm`. Два стека совместимы: 2020 тестов зелёные
Слои GeoJSON в состоянии попадают в `state_delta` событий сессии	Раздувание Redis или БД. Сейчас чекпоинты тоже хранят слои, но один раз	Хранить слои в `ArtifactService`, в состоянии — только ссылки
Другая семантика возобновления: сессия ADK вместо буфера по `request_id`, своя приостановка по токену	Переподключение нужно делать заново	Повторять события сессии через `LegacyEventAdapter`. Проверить на втором пилоте
Накладные расходы движка — примерно +45 мс на запрос (17 узлов, in-memory сессия)	Низкое на фоне секунд LLM. С Redis-сессией будет больше	Замерить на реальном стеке

Риск	Влияние	Что делать
Неочевидности графа: нет ключа в state — сразу ошибка; `return` в генераторе молча теряет выход; LLM-узел берёт вход предыдущего узла как сообщение пользователя	Тонкие ошибки при переносе	Использовать паттерны пилота: callback `conversation`, явные ключи state и тесты эквивалентности
Точность на реальных моделях не замерялась	Низкое: запросы совпадают дословно	Прогнать `evaluation/orchestrator` на обоих движках за флагом

Открытые вопросы

- [ ] Планирует ли Synapse переход на A2A 1.0? От этого зависит, когда переходить на встроенный A2A из ADK.
- [ ] Где хранить сессии ADK: в текущем Redis (`RedisSessionService`) или в Postgres (`DatabaseSessionService`)?
- [ ] Нужен ли дальше Ollama или достаточно OpenAI-совместимых эндпоинтов? `GmartLlm` поддерживает оба варианта.
- [ ] Делать ли уточнение паузой HITL (`RequestInput`) с возобновлением? Это меняет контракт фронта, поэтому не раньше, чем после миграции.

План полной миграции и оценка трудозатрат

Полный переход на идиомы ADK займёт примерно 16–20 недель одного разработчика, или 10–12 недель двоих. Этапы идут по одному агенту за флагом, с теми же тестами эквивалентности, что в пилоте. Оценка экспертная: она опирается на объём кода и на то, что показал пилот. В нём каждый LLM-шаг стал одним `LlmAgent`, каждое ветвление — одним маршрутом, а доменная логика не изменилась.

Этап	Что входит	Оценка	Критерий готовности
0. Общий слой	`GmartLlm` в `src/`, учёт токенов, `llm_pace` и `context_budget` как плагины, `LegacyEventAdapter`, сессии в Redis, слои в `ArtifactService`, тесты эквивалентности в CI	2 нед.	Пилот работает на общем слое, CI зелёный
1. Restriction в прод	Узел NormGraph, запись чата, повтор событий при переподключении, флаг в `dependencies.py`	2 нед.	`evaluation/orchestrator` даёт не хуже текущего, задержка в пределах +5%
2. Малые агенты	pzz (~1 тыс. строк), provision (~2,3 тыс.), normgraph (~1,9 тыс.)	3 нед.	Тесты эквивалентности и A2A на каждый агент
3. Большие агенты	dvd (~7,6 тыс., петли critic и revision), scenario_data (~10,9 тыс., 7 LLM-шагов, 10 MCP-вызовов), compliance (~8 тыс., диалоги выбора и сопоставления)	6–9 нед.	Качество на `evaluation/documents_*` не ниже текущего
4. Оркестратор	План шагов через вложенные workflow, `ctx.run_node()` и `ParallelWorker` вместо своих `_run_steps` и `_run_ahead`	2 нед.	Смешанные сценарии из `evaluation/orchestrator` проходят
5. Уборка	Удалить старый движок, чекпоинты `PipelineStateStore`, дубли промптов. Портировать unit-тесты	1–2 нед.	Один движок, тесты зелёные

Что из ADK брать сразу: `Workflow`, `LlmAgent` с `output_schema`, callbacks, plugins, `RetryConfig`, сессии и оценку качества. Что не брать сейчас: `OpenAILlm` и `LiteLlm`, встроенный A2A (пока Synapse на 0.3), `McpToolset` с выбором инструментов моделью для детерминированных пайплайнов, dev UI.

Источники

- google-adk 2.13.0 на PyPI: исследованная версия и список релизов.
- google/adk-python, README: Agent и Workflow, ссылки на документацию.
- google/adk-python, docs/guides на теге v2.13.0: Workflow, FunctionNode, Event, LlmAgent single-turn, ReflectAndRetryModelPlugin, A2A.
- google/adk-python, contributing/samples: workflows/route, loop, retry, agent_in_workflow, models/hello_world_openai.
- a2a-sdk: протокол 1.0 и режим совместимости с 0.3.
- Исходники gMART (ветка `dev`) и пилот в ветке `research/adk-restriction-pilot`, папка `research/adk_pilot/`.